TheoInf1 Woche 11

▸ Lösungen zur Hausübung 3

▸ Aufgabe 1: Extended Loop ⇒ Sprache [4+4 Punkte]

Sei Σ = {A, B, C} das Alphabet. Definieren Sie zu jedem der beiden folgenden Extended Loop-Programme die Sprache L ⊆ Σ*, die dieses Programm akzeptiert! Verwenden Sie dabei die Notation "L = {…}"!

1.

```
n ← len(in)
if n % 2
    return false
end
h ← n / 2
i ← 0
j ← h
loop h
    if in[i] ≠ in[j]
        return false
    end
    ++i
    ++j
end
return true
```

L = {ww | w ∈ Σ*}
  = {w ∈ Σ* | es existiert ein u ∈ Σ* mit w = uu}

2.

```
n ← len(in)
i ← 0
ca ← 0
cb ← 0
for i ← 0, …, n−1
    if in[i] = "A"
        ++ca
    end
    if in[i] = "B"
        ++cb
    end
end
if ca = cb
    return true
else
    return false
end
```

L = {w ∈ Σ* | w enthält genauso viele A wie B}

▶ **Aufgabe 2: Extended Loop programmieren** [6 Punkte]

Sei Σ = {A, B, C} das Alphabet. Schreiben Sie zu der folgenden Sprachen L ⊆ Σ* ein Programm in der Programmiersprache "*extended Loop*", das genau diese Sprache akzeptiert:

$$L = \{A^k BC^{k!} \mid k \in \mathbb{N}\}$$

Dabei ist k! – die *Fakultät von k* – das Produkt der Zahlen von 1 bis n, also k! = 1 · 2 · … · k. Ihr Programm soll *true* zurückgeben, wenn die Eingabevariable einen String aus der Sprache kodiert; andernfalls soll Ihr Programm *false* zurückgeben. Ihr Loop-Programm erhält als Eingabe in der Variablen "in" einen String aus Σ*, es muss also nicht prüfen, ob der Eingabestring Zeichen ∉ Σ enthält.

Lösungsvorschlag:

```
n ← len(in)
k ← 0
loop n
    if in[k] = "A"
        ++k
    end
end
if (k = 0) or (in[k] ≠ B)
    return false
end
i ← k + 1
fac ← 1
loop k
    fac ← fac · k
    −−k
end
if i + fac ≠ n
    return false
end
loop fac
    if in[i] ≠ "C"
        return false
    end
    ++i
end
return true
```

A zählen und k bestimmen

k ∈ ℕ, also ≠ 0. Prüfe, ob an Position k ein B steht

k+1 in i merken

berechne k!

prüfe, ob der String die korrekte Länge hat

prüfe, ob am Ende k! viele C stehen

alles klar, der String wird akzeptiert

▶ **Aufgabe 3: Die While-Sprache** [1 + 1 + 1 + 3 Punkte]

Betrachten Sie folgendes Extended-While-Programm, das in der Variablen *in* einen String aus dem Alphabet Σ = {A, B, C} als Eingabe bekommt:

```
n ← len(in)
i ← n−1
while in[i] ≠ "B"
   if in[i] = "A"
      return true
   end
   −−i
end
return false
```

1. Geben Sie ein Beispiel für einen Eingabestring, den das Programm **akzeptiert**, d.h. für den das Programm den Wert *true* zurückgibt!

   z.B. in = "A" oder auch in = "CBBABC**A**CCC"

2. Geben Sie ein Beispiel für einen Eingabestring, den das Programm **ablehnt**, d.h. für den das Programm den Wert *false* zurückgibt!

   z.B. in = "B" oder auch in = "ACBBACCA**B**C"

3. Geben Sie ein Beispiel für einen Eingabestring, bei dem sich das Programm "**aufhängt**", d.h. für den die durch das Programm berechnete Funktion *nicht definiert* ist!

   z.B. in = "C" oder auch in = "CCCC" oder auch in = ε

4. Schreiben Sie ein Programm in der Programmiersprache **Extended Loop**, das dieselbe Sprache akzeptiert, wie das oben aufgeführte Extended-While-Programm! Beide Programme sollen also für genau dieselben Eingabestrings den Wert *true* zurückgeben.

```
n ← len(in)
i ← n−1
loop n
    if in[i] = "B"
        return false
    end
    if in[i] = "A"
        return true
    end
    −−i
end
return false
```

Das letzte "*return false*" kann man auch weglassen; man könnte das Programm auch mit dem Befehl "*hangup*" abschließen.

(weitere Übungsaufgaben)

▸ Aufgabe 1: Mit Extended Loop rechnen

1. Schreiben Sie ein *Extended Loop*-Programm, das zwei Zahl n, m ∈ $\mathbb{N}_0$ als Eingabe bekommt und *true* zurückgibt, wenn n durch m ohne Rest teilbar ist, und das ansonsten *false* zurückgibt.

```
i ← n / m
return i · m = n
```

oder:

```
return n % m = 0
```

oder auch:

```
loop n
    if n ≥ m
        n ← n − m
    end
end
return n
```

2. Schreiben Sie ein *Extended Loop*-Programm, das eine Zahl n ∈ $\mathbb{N}$ als Eingabe bekommt, und *true* zurück gibt, wenn n eine Primzahl ist, und das ansonsten *false* zurück gibt.

```
if n ≤ 1
    return false
end
i ← 2
loop n−2
    if n % i = 0
        return false
    end
    ++i
end
return true
```

oder auch

```
if n ≤ 1
    return false
end
if n = 2
    return true
end
for i ← 2, …, n−1
    if n % i = 0
        return false
    end
end
return true
```

▸ Aufgabe 2: Mit Extended While abstürzen

Schreiben Sie ein *Extended While*-Programm, dass nur definiert ist, wenn seine Eingabe n $\in \mathbb{N}_0$ kleiner als 10 ist (n < 10). In diesem Fall soll das Programm mit "*return true*" stoppen. (Andernfalls soll es abstürzen.)

```
if n < 10
    return true
end
while true
end
```

▸ Aufgabe 3: Listen in Loop

In der Vorlesung wurde besprochen, wie man in einer einzelnen Variablen a eine Liste $[a_0, a_1, a_2, \ldots, a_{n-1}]$ von Zahlen aus $\mathbb{N}_0$ speichern kann. Schreiben Sie ein *Extended-loop*-Programm, das eine Liste a als Eingabe bekommt und prüft, ob die Zahlen in der Liste aufsteigend sortiert sind. Um die Anzahl der in der Liste a gespeicherten Zahlen zu bekommen, verwenden Sie die Funktion *count*(a); das i-te Element von a erhalten Sie durch den Ausdruck a_i.

```
n ← count(a)
if n ≤ 1
    return true
end
for i ← 0, …, n−2
    if a_i > a_{i+1}
        return false
    end
end
return true
```